

PANNs: Large-Scale Pretrained Audio Neural Networks for Audio Pattern Recognition

Qiuqiang Kong, *Student Member, IEEE*, Yin Cao, *Member, IEEE*, Turab Iqbal, Yuxuan Wang, Wenwu Wang, *Senior Member, IEEE* and Mark D. Plumbley, *Fellow, IEEE*

Abstract—Audio pattern recognition is an important research topic in the machine learning area, and includes several tasks such as audio tagging, acoustic scene classification, music classification, speech emotion classification and sound event detection. Recently, neural networks have been applied to tackle audio pattern recognition problems. However, previous systems are built on specific datasets with limited durations. Recently, in computer vision and natural language processing, systems pretrained on large-scale datasets have generalized well to several tasks. However, there is limited research on pretraining systems on large-scale datasets for audio pattern recognition. In this paper, we propose pretrained audio neural networks (PANNs) trained on the large-scale AudioSet dataset. These PANNs are transferred to other audio related tasks. We investigate the performance and computational complexity of PANNs modeled by a variety of convolutional neural networks. We propose an architecture called Wavegram-Logmel-CNN using both log-mel spectrogram and waveform as input feature. Our best PANN system achieves a state-of-the-art mean average precision (mAP) of 0.439 on AudioSet tagging, outperforming the best previous system of 0.392. We transfer PANNs to six audio pattern recognition tasks, and demonstrate state-of-the-art performance in several of those tasks. We have released the source code and pretrained models of PANNs: https://github.com/qiuqiangkong/audioset_tagging_cnn.

Index Terms—Audio tagging, pretrained audio neural networks, transfer learning.

I. INTRODUCTION

Audio pattern recognition is an important research topic in the machine learning area, and plays an important role in our life. We are surrounded by sounds that contain rich information of where we are, and what events are happening around us. Audio pattern recognition contains several tasks such as audio tagging [1], acoustic scene classification [2], music classification [3], speech emotion classification and sound event detection [4].

Audio pattern recognition has attracted increasing research interest in recent years. Early audio pattern recognition work

focused on private datasets collected by individual researchers [5][6]. For example, Woodard [5] applied a hidden Markov model (HMM) to classify three types of sounds: wooden door open and shut, dropped metal and poured water. Recently, the Detection and Classification of Acoustic Scenes and Events (DCASE) challenge series [7][8][9][2] have provided publicly available datasets, such as acoustic scene classification and sound event detection datasets. The DCASE challenges have attracted increasing research interest in audio pattern recognition. For example, the recent DCASE 2019 challenge received 311 entries across five subtasks [10].

However, it is still an open question how well an audio pattern recognition system can perform when trained on large-scale datasets. In computer vision, several image classification systems have been built with the large-scale ImageNet dataset [11]. In natural language processing, several language models have been built with the large-scale text datasets such as Wikipedia [12]. However, systems trained on large-scale audio datasets have been more limited [1][13][14][15].

A milestone for audio pattern recognition was the release of AudioSet [1], a dataset containing over 5,000 hours of audio recordings with 527 sound classes. Instead of releasing the raw audio recordings, AudioSet released embedding features of audio clips extracted from a pretrained convolutional neural network [13]. Several researchers have investigated building systems with those embedding features [13][16][17][18][19][20]. However, the embedding features may not be an optimal representation for audio recordings, which may limit the performance of those systems. In this article, we propose pretrained audio neural networks (PANNs) trained on raw AudioSet recordings with a wide range of neural networks. We show that several PANN systems outperform previous state-of-the-art audio tagging systems. We also investigate the audio tagging performance and computation complexities of PANNs.

We propose that PANNs can be transferred to other audio pattern recognition tasks. Previous researchers have previously investigated transfer learning for audio tagging. For example, audio tagging systems were pretrained on the Million Song Dataset were proposed in [21], with embedding features extracted from pretrained convolutional neural networks (CNNs) are used as inputs to second-stage classifiers such as neural networks or support vector machines (SVMs) [14][22]. Systems pretrained on MagnaTagATune [23] and acoustic scene [24] datasets were fine-tuned on other audio tagging tasks [25][26]. These transfer learning systems were mainly trained with music datasets, and were limited to smaller datasets than AudioSet.

The contribution of this work includes: (1) We introduce

Q. Kong, Y. Cao, T. Iqbal, and M. D. Plumbley are with the Centre for Vision, Speech and Signal Processing, University of Surrey, Guildford GU2 7XH, U.K. (e-mail: q.kong@surrey.ac.uk; yin.cao@surrey.ac.uk; t.iqbal@surrey.ac.uk; m.plumbley@surrey.ac.uk).

This work was supported in part by the EPSRC Grant EP/N014111/1 “Making Sense of Sounds”, in part by the Research Scholarship from the China Scholarship Council 201406150082, and in part by a studentship (Reference: 1976218) from the EPSRC Doctoral Training Partnership under Grant EP/N509772/1. This work was supported by National Natural Science Foundation of China (Grant No. 11804365). (Qiuqiang Kong is first author.) (Yin Cao is corresponding author.)

Y. Wang is with the ByteDance AI Lab, Mountain View, CA, USA (e-mail: wangyuxuan.11@bytedance.com).

W. Wang is with the Centre for Vision, Speech and Signal Processing, University of Surrey, Guildford GU2 7XH, U.K., and also with Qingdao University of Science and Technology, Qingdao 266071, China (e-mail: w.wang@surrey.ac.uk).

PANNs trained on AudioSet with 1.9 million audio clips with an ontology of 527 sound classes; (2) We investigate the trade-off between audio tagging performance and computation complexity of a wide range of PANNs; (3) We propose a system that we call Wavegram-Logmel-CNN that achieves a mean average precision (mAP) of 0.439 on AudioSet tagging, outperforming previous state-of-the-art system with an mAP 0.392 and Google’s system with an mAP 0.314; (4) We show that PANNs can be transferred to other audio pattern recognition tasks, outperforming several state-of-the-art systems; (5) We have released the source code and pretrained PANN models.

This paper is organized as follows: Section II introduces audio tagging with various convolutional neural networks; Section III introduces our proposed Wavegram-CNN systems; Section IV introduces our data processing techniques, including data balancing and data augmentation for AudioSet tagging; Section VI shows experimental results, and Section VII concludes this work.

II. AUDIO TAGGING SYSTEMS

Audio tagging is an essential task of audio pattern recognition, with the goal of predicting the presence or absence of audio tags in an audio clip. Early work on audio tagging included using manually-designed features as input, such as audio energy, zero-crossing rate, and mel-frequency cepstrum coefficients (MFCCs) [27]. Generative models, including Gaussian mixture models (GMMs) [28][29], hidden Markov models (HMMs), and discriminative support vector machines (SVMs) [30] have been used as classifiers. Recently, neural network based methods such as convolutional neural networks (CNNs) have been used [3] to predict the tags of audio recordings. CNN-based systems have achieved state-of-the-art performance in several DCASE challenge tasks including acoustic scene classification [2] and sound event detection [4]. However, many of those works focused on particular tasks with a limited number of sound classes, and were not designed to recognize a wide range of sound classes. In this article, we focus on training large-scale PANNs on AudioSet [1] to tackle the general audio tagging problem.

A. CNNs

1) *Conventional CNNs*: CNNs have been successfully applied to computer vision tasks such as image classification [31][32]. A CNN consists of several convolutional layers. Each convolutional layer contains several kernels that are convolved with the input feature maps to capture their local patterns. CNNs adopted for audio tagging [3][20] often use log mel spectrograms as input [3][20]. Short time Fourier transforms (STFTs) are applied to time-domain waveforms to calculate spectrograms. Then, mel filter banks are applied to the spectrograms, followed by a logarithmic operation to extract log mel spectrograms [3][20].

2) *Adapting CNNs for AudioSet tagging*: The PANNs we use are based on our previously-proposed cross-task CNN systems for the DCASE 2019 challenge [33], with an extra fully-connected layer added to the penultimate layer of CNNs

TABLE I
CNNs FOR AUDIOSET TAGGING

VGGish [1]	CNN6	CNN10	CNN14
Log-mel spectrogram 96 frames $\times$ 64 mel bins	Log-mel spectrogram 1000 frames $\times$ 64 mel bins		
$3 \times 3 @ 64$ ReLU	$5 \times 5 @ 64$ BN, ReLU	$(3 \times 3 @ 64) \times 2$ BN, ReLU	$(3 \times 3 @ 64) \times 2$ BN, ReLU
MP 2×2	Pooling 2×2		
$3 \times 3 @ 128$ ReLU	$5 \times 5 @ 128$ BN, ReLU	$(3 \times 3 @ 128) \times 2$ BN, ReLU	$(3 \times 3 @ 128) \times 2$ BN, ReLU
MP 2×2	Pooling 2×2		
$(3 \times 3 @ 256) \times 2$ ReLU	$5 \times 5 @ 256$ BN, ReLU	$(3 \times 3 @ 256) \times 2$ BN, ReLU	$(3 \times 3 @ 256) \times 2$ BN, ReLU
MP 2×2	Pooling 2×2		
$(3 \times 3 @ 512) \times 2$ ReLU	$5 \times 5 @ 512$ BN, ReLU	$(3 \times 3 @ 512) \times 2$ BN, ReLU	$(3 \times 3 @ 512) \times 2$ BN, ReLU
MP 2×2 Flatten	Global pooling		
FC 4096 ReLU $\times 2$	FC 512, ReLU		
FC 527, Sigmoid	FC 527, Sigmoid		
			Pooling 2×2
			$(3 \times 3 @ 1024) \times 2$ BN, ReLU
			$(3 \times 3 @ 2048) \times 2$ BN, ReLU
			Global pooling
			FC 2048, ReLU
			FC 527, Sigmoid

to further increase the representation ability. We investigate 6-, 10- and 14-layer CNNs. The 6-layer CNN consists of 4 convolutional layers with a kernel size of 5×5 , based on AlexNet [34]. The 10- and 14-layer CNNs consist of 4 and 6 convolutional layers, respectively, inspired by the VGG-like CNNs [35]. Each convolutional block consists of 2 convolutional layers with a kernel size of 3×3 . Batch normalization [36] is applied between each convolutional layer, and the ReLU nonlinearity [37] is used to speed up and stabilize the training. We apply average pooling of size of 2×2 to each convolutional block for downsampling, as 2×2 average pooling has been shown to outperform 2×2 max pooling [33].

Global pooling is applied after the last convolutional layer to summarize the feature maps into a fixed-length vector. In [15], maximum and average operation were used for global pooling. To combine their advantages, we sum the averaged and maximized vectors. In our previous work [33], those fixed-length vectors were used as embedding features for audio clips. In this work, we add an extra fully-connected layer to the fixed length vectors to extract embedding features which can further increase their representation ability. For a particular audio pattern recognition task, a linear classifier is applied to the embedding features, followed by either a softmax nonlinearity for classification tasks or a sigmoid nonlinearity for tagging tasks. Dropout [38] is applied after each downsampling operation and fully connected layers to prevent systems from overfitting. Table I summarizes our proposed CNN systems. The number after the “@” symbol indicates the number of feature maps. The first column shows the VGGish network proposed by [13]. MP is the abbreviation of max pooling. The “Pooling 2×2 ” in Table I is average pooling with size of 2×2 . In [13], an audio clip was split into 1-second segments, [13] also assumed each segment inherits the label of the audio clip, which may lead to incorrect labels. In contrast, our systems from the second to the fourth columns in Table I applies an entire audio clip for training without cutting the audio clip into segments.

TABLE II
RESNETS FOR AUDIOSET TAGGING

ResNet22	ResNet38	ResNet54
Log mel spectrogram 1000 frames $\times$ 64 mel bins		
$(3 \times 3 @ 512, \text{BN}, \text{ReLU}) \times 2$		
Pooling 2×2		
$(\text{BasicB} @ 64) \times 2$	$(\text{BasicB} @ 64) \times 3$	$(\text{BottleneckB} @ 64) \times 3$
Pooling 2×2		
$(\text{BasicB} @ 128) \times 2$	$(\text{BasicB} @ 128) \times 4$	$(\text{BottleneckB} @ 128) \times 4$
Pooling 2×2		
$(\text{BasicB} @ 256) \times 2$	$(\text{BasicB} @ 256) \times 6$	$(\text{BottleneckB} @ 256) \times 6$
Pooling 2×2		
$(\text{BasicB} @ 512) \times 2$	$(\text{BasicB} @ 512) \times 3$	$(\text{BottleneckB} @ 512) \times 3$
Pooling 2×2		
$(3 \times 3 @ 2048, \text{BN}, \text{ReLU}) \times 2$		
Global pooling		
FC 2048, ReLU		
FC 527, Sigmoid		

We denote the waveform of an audio clip as x_n , where n is the index of audio clips, and $f(x_n) \in [0, 1]^K$ is the output of a PANN representing the presence probabilities of K sound classes. The label of x_n is denoted as $y_n \in \{0, 1\}^K$. A binary cross-entropy loss function l is used to train a PANN:

$$l = - \sum_{n=1}^N (y_n \cdot \ln f(x_n) + (1 - y_n) \cdot \ln(1 - f(x_n))), \quad (1)$$

where N is the number of training clips in AudioSet. In training, the parameters of $f(\cdot)$ are optimized by using gradient descent methods to minimize the loss function l .

B. ResNets

1) *Conventional residual networks (ResNets)*: Deeper CNNs have been shown to achieve better performance than shallower CNNs for audio classification [31]. One challenge of very deep conventional CNNs is that the gradients do not propagate properly from the top layers to the bottom layers [32]. To address this problem, ResNets [32] introduced shortcut connections between convolutional layers. In this way, the forward and backward signals can be propagated from one layer to any other layer directly. The shortcut connections only introduce a small number of extra parameters and a little additional computational complexity. A ResNet consists of several blocks, where each block consists of two convolutional layers with a kernel size of 3×3 , and a shortcut connection between input and output. Each bottleneck block consists of three convolutional layers with a network-in-network architecture [39] that can be used instead of the basic blocks in a ResNet [32].

2) *Adapting ResNets for AudioSet tagging*: We adapt ResNet [32] for AudioSet tagging as follows. To begin with, two convolutional layers and a downsampling layer are applied on the log mel spectrogram to reduce the input log mel spectrogram size. We implement three types of ResNets with different depths: a 22-layer ResNet with 8 basic blocks; a 38-layer ResNet with 16 basic blocks, and a 54-layer ResNet with 16 residual blocks. Table II shows the architecture of the ResNet systems adapted for AudioSet tagging. The BasicB and BottleneckB are abbreviations of basic block and bottleneck block, respectively.

TABLE III
MOBILENETS FOR AUDIOSET TAGGING

MobileNetV1	MobileNetV2
$3 \times 3 @ 32, \text{BN}, \text{ReLU}$	
Pooling 2×2	
V1Block @ 64 V1Block @ 128	V2Block, t=1 @ 16 (V2Block, t=6 @ 24) $\times$ 2
Pooling 2×2	
V1Block @ 128 V1Block @ 256	(V2Block, t=6 @ 32) $\times$ 3
Pooling 2×2	
V1Block @ 256 V1Block @ 512	(V2Block, t=6 @ 64) $\times$ 4
Pooling 2×2	
(V1Block @ 512) $\times$ 5 V1Block @ 1024	(V2Block, t=6 @ 96) $\times$ 3
Pooling 2×2	
V1Block @ 1024	(V2Block, t=6 @ 160) $\times$ 3 (V2Block, t=6 @ 320) $\times$ 1
Global pooling	
FC, 1024, ReLU	
FC, 527, Sigmoid	

C. MobileNets

1) *Conventional MobileNets*: Computational complexity is an important issue when systems are implemented on portable devices. Compared to CNNs and ResNets, MobileNets were intended to reduce the number of parameters and multiply-add operations in a CNN. MobileNets were based on depthwise separable convolutions [40] by factorizing a standard convolution into a depthwise convolution and a 1×1 pointwise convolution [40].

2) *Adapting MobileNets for AudioSet tagging*: We adapt MobileNetV1 [40] and MobileNetV2 [41] systems for AudioSet tagging shown in Table III. The V1Blocks and V2Blocks are MobileNet convolutional blocks [40][41], each consisting of two and three convolutional layers, respectively.

D. One-dimensional CNNs

Previous audio tagging systems were based on the log mel spectrogram, a hand-crafted feature. To improve performance, several researchers proposed to build one-dimensional CNNs which operate directly on the time-domain waveforms. For example, Dai et al. [31] proposed a one-dimensional CNN for acoustic scene classification, and Lee et al. [42] proposed a one-dimensional CNN that was later adopted by Pons et al. [15] for music tagging.

1) *DaiNet*: DaiNet [31] applied kernels of length 80 and stride 4 to the input waveform of audio recordings. The kernels are learnable during training. To begin with, a maximum operation is applied to the first convolutional layer, which is designed to make the system be robust to phase shift of the input signals. Then, several one-dimensional convolutional blocks with kernel size 3 and stride 4 were applied to extract high level features. An 18-layer DaiNet with four convolutional layers in each convolutional block achieved the best result in UrbanSound8K [43] classification [31].

2) *LeeNet*: In contrast to DaiNet that applied large kernels in the first layer, LeeNet [42] applied small kernels with length 3 on the waveforms, to replace the STFT for spectrogram

extraction. LeeNet consists of several one dimensional convolutional layers, each followed by a downsampling layer of size 2. The original LeeNet consists of 11 layers.

3) *Adapting one-dimensional CNNs for AudioSet tagging:* We modify LeeNet by extending it to a deeper architecture with 24 layers, replacing each convolutional layer with a convolutional block that consists of two convolutional layers. To further increase the number of layers of the one-dimensional CNNs, we propose a one-dimensional residual network (Res1dNet) with a small kernel size of 3. We replace the convolutional blocks in LeeNet with residual blocks, where each residual block consists of two convolutional layers with kernel size 3. The first and second convolutional layers of convolutional block have dilations of 1 and 2, respectively, to increase the receptive field of the corresponding residual block. Downsampling is applied after each residual block. By using 14 and 24 residual blocks, we obtain a Res1dNet31 and a Res1dNet51 with 31 and 51 layers, respectively.

III. WAVEGRAM-CNN SYSTEMS

Previous one-dimensional CNN systems [31][42][15] have not outperformed the systems trained with log mel spectrograms as input. One characteristic of previous time-domain CNN systems [31][42] is that they were not designed to capture frequency information, because there is no frequency axis in the one-dimensional CNN systems, so they can not capture frequency patterns of a sound event with different pitch shifts.

A. Wavegram-CNN systems

In this section, we propose architectures which we call Wavegram-CNN and Wavegram-Logmel-CNN for AudioSet tagging. The Wavegram-CNN we propose is a time-domain audio tagging system. Wavegram is a feature we propose that is similar to log mel spectrogram, but is learnt using a neural network. A Wavegram is designed to learn a time-frequency representation that is a modification of the Fourier transform. A Wavegram has a time axis and a frequency axis. Frequency patterns are important for audio pattern recognition, for example, sounds with different pitch shifts belong to the same class. A Wavegram is designed to learn frequency information that may be lacking in one-dimensional CNN systems. Wavegrams may also improve over hand-crafted log mel spectrograms by learning a new kind of time-frequency transform from data. Wavegrams can then replace log mel spectrograms as input features resulting in our Wavegram-CNN system. We also combine the Wavegram and the log mel spectrogram as a new feature to build the Wavegram-Logmel-CNN system as shown in Fig. 1.

To build a Wavegram, we first apply a one-dimensional CNN to time-domain waveform. The one-dimensional CNN begins with a convolutional layer with filter length 11 and stride 5 to reduce the size of the input. This immediately reduces the input lengths by a factor of 5 times to reduce memory usage. This is followed by three convolutional blocks, where each convolutional block consists of two convolutional layers with dilations of 1 and 2, respectively, which are

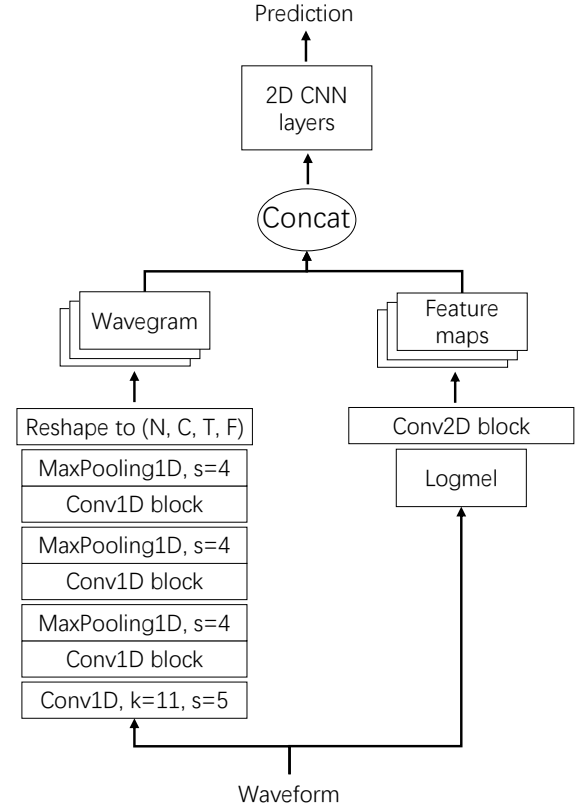


Fig. 1. Architecture of Wavegram-Logmel-CNN

designed to increase the receptive field of the convolutional layers. Each convolutional block is followed by a downsampling layer with stride 4. By using the stride and downsampling three times, we downsample a 32 kHz audio recording to $32,000/5/4/4/4 = 100$ frames of features per second. We denote the output size of the one-dimensional CNN layers as $T \times C$, where T is the number of frames and C is the number of channels. We reshape this output to a tensor with a size of $T \times F \times C/F$ by splitting C channels into C/F groups, where each group has F frequency bins. We call this tensor a *Wavegram*. The Wavegram learns frequency information by introducing F frequency bins in each of C/F channels. We apply CNN14 described in Section II-A as a backbone architecture on the extracted Wavegram, so that we can fairly compare the Wavegram and log mel spectrogram based systems. Two dimensional CNNs such as CNN14 can capture time-frequency invariant patterns on the Wavegram, because kernels are convolved along both time and frequency axis in a Wavegram.

B. Wavegram-Logmel-CNN

Furthermore, we can combine the Wavegram and log mel spectrogram into a new representation. In this way, we can utilize the information from both time-domain waveforms and log mel spectrograms. The combination is carried out along the channel dimension. The Wavegram provides extra information for audio tagging, complementing the log mel spectrogram. Fig. 1 shows the architecture of the Wavegram-Logmel-CNN.

IV. DATA PROCESSING

In this section, we introduce data processing for AudioSet tagging, including data balancing and data augmentation. Data balancing is a technique used to train neural networks on a highly unbalanced dataset. Data augmentation is a technique used to augment the dataset, to prevent systems from overfitting during training.

A. Data balancing

The number of audio clips available for training varies from sound class to sound class. For example, there are over 900,000 audio clips belonging to the categories “Speech” and “Music”. On the other hand, there are only tens of audio clips belonging to the category “Toothbrush”. The number of audio clips of different sound classes has a long tailed distribution. Training data are input to a PANN in mini-batches during training. Without a data balancing strategy, audio clips are uniformly sampled from AudioSet. Therefore, sound classes with more training clips such as “Speech” are more likely to be sampled during training. In an extreme case, all data in a mini-batch may belong to the same sound class. This will cause the PANN to overfit to sound classes with more training clips, and underfit to sound classes with fewer training clips. To solve this problem, we design a balanced sampling strategy to train PANNs. That is, audio clips are approximately equally sampled from all sound classes to constitute a mini-batch. We use the term “approximately” because an audio clip may contain more than one tag.

B. Data augmentation

Data augmentation is a useful way to prevent a system from overfitting. Some sound classes in AudioSet contain only a small number (e.g., hundreds) of training clips which may limit the performance of PANNs. We apply mixup [44] and SpecAugment [45] to augment data during training.

1) *Mixup*: Mixup [44] is a way to augment a dataset by interpolating both the input and target of two audio clips from a dataset. For example, we denote the input of two audio clips as x_1, x_2 , and their targets as y_1, y_2 , respectively. Then, the augmented input and target can be obtained by $x = \lambda x_1 + (1 - \lambda)x_2$ and $y = \lambda y_1 + (1 - \lambda)y_2$ respectively, where λ is sampled from a Beta distribution [44]. By default, we apply mixup on the log mel spectrogram. We will compare the performance of mixup augmentation on the log mel spectrogram and on the time-domain waveform in Section VI-C4.

2) *SpecAugment*: SpecAugment [45] was proposed for augmenting speech data for speech recognition. SpecAugment operates on the log mel spectrogram of an audio clip using frequency masking and time masking. Frequency masking is applied such that f consecutive mel frequency bins $[f_0, f_0 + f]$ are masked, where f is chosen from a uniform distribution from 0 to a frequency mask parameter f' , and f_0 is chosen from $[0, F - f]$, where F is the number of mel frequency bins [45]. There can be more than one frequency mask in each log mel spectrogram. The frequency mask can improve the robustness of PANNs to frequency distortion of audio clips [45]. Time masking is similar to frequency masking, but is applied in the time domain.

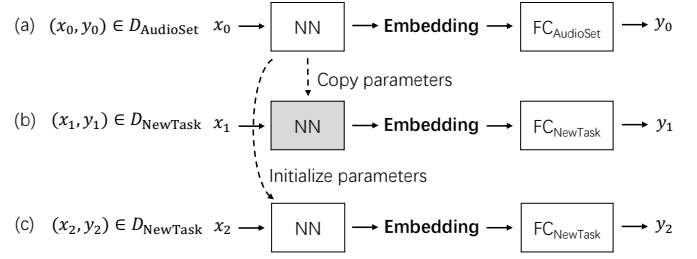


Fig. 2. (a) A PANN is pretrained with the AudioSet dataset. (b) For a new task, the PANN is used as a feature extractor. A classifier is built on the extracted embedding features. The shaded rectangle indicates the parameters are frozen and not trained. (c) For a new task, the parameters of a neural network are initialized with a PANN. Then, all parameters are fine-tuned on the new task.

V. TRANSFER TO OTHER TASKS

To investigate the generalization ability of PANNs, we transfer PANNs to a range of audio pattern recognition tasks. Previous works on audio transfer learning [21][14][22][25][23] mainly focused on music tagging, and were limited to smaller datasets than AudioSet. To begin with, we demonstrate the training of a PANN in Fig. 2(a). Here, D_{AudioSet} is the AudioSet dataset, and x_0, y_0 are training input and target, respectively. FC_{AudioSet} is the fully connected layer for AudioSet tagging. In this article, we propose to compare the following transfer learning strategies.

1) Train a system from scratch. All parameters are randomly initialized. Systems are similar to PANNs, except for the final fully-connected layer which depends on the task dependent number of outputs. This system is used as a baseline system to be compared with other transfer learning systems.

2) Use a PANN as a feature extractor. For new tasks, the embedding features of audio clips are calculated by using the PANN. Then, the embedding features are used as input to a classifier, such as a fully-connected neural network. When training on new tasks, the parameters of the PANN are frozen and not trained. Only the parameters of the classifier built on the embedding features are trained. Fig. 2(b) shows this strategy, where D_{NewTask} is a new task dataset, and FC_{NewTask} is the fully connected layer of a new task. The PANN is used as a feature extractor. A classifier is built on the extracted embedding features. The shaded rectangle indicates the parameters which are frozen and not trained.

(3) Fine-tune a PANN. A PANN is used for a new task, except the final fully-connected layer. All parameters are initialized from the PANN, except the final fully-connected layer which is randomly initialized. All parameters are fine-tuned on D_{NewTask} . Fig. 2(c) demonstrates the fine-tuning of a PANN.

VI. EXPERIMENTS

First, we evaluate the performance of PANNs on AudioSet tagging. Then, the PANNs are transferred to several audio pattern recognition tasks, including acoustic scene classification, general audio tagging, music classification and speech emotion classification.

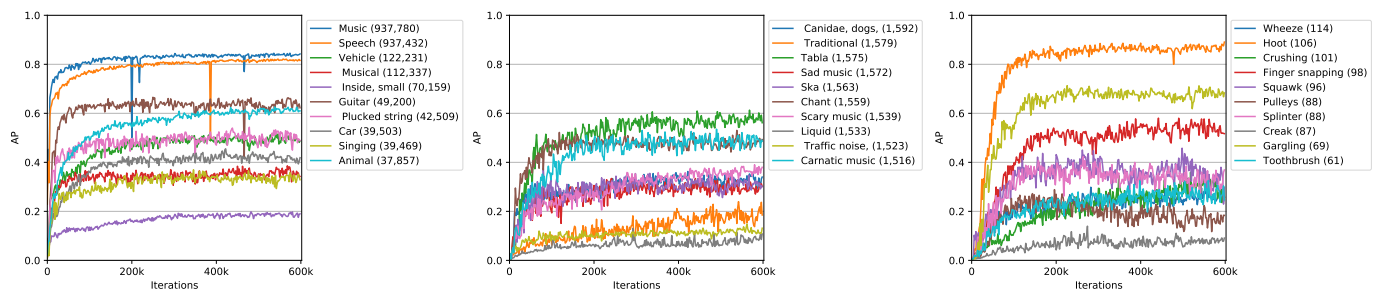


Fig. 3. Class-wise AP of sound events with the CNN14 system. The number inside parentheses indicates the number of training clips. The left, middle, right columns show the AP of sound classes with the number of training clips ranked the 1st to 10th, 250th to 260th and 517th to 527th in the training set of AudioSet.

A. AudioSet dataset

AudioSet is a large-scale audio dataset with an ontology of 527 sound classes [1]. The audio clips from AudioSet are extracted from YouTube videos. The training set consists of 2,063,839 audio clips, including a “balanced subset” of 22,160 audio clips, where there are at least 50 audio clips for each sound class. The evaluation set consists of 20,371 audio clips. Instead of using the embedding features provided by [1], we downloaded raw audio waveforms of AudioSet in Dec. 2018 using the links provided by [1], and ignored the audio clips that are no longer downloadable. We successfully download 1,934,187 (94%) of the audio clips of the full training set, including 20,550 (93%) of the audio clips of the balanced training set. We successfully download 18,887 audio clips of the evaluation dataset. We pad the audio clips to 10 seconds with silence if they are shorter than 10 seconds. Considering the fact that a large number of audio clips from YouTube are monophonic and have a low sampling rate, we convert all audio clips to monophonic and resample them to 32 kHz.

For the CNN systems based on log mel spectrograms, STFT is applied on the waveforms with a Hamming window of size 1024 [33] and a hop size of 320 samples. This configuration leads to 100 frames per second. Following [33], we apply 64 mel filter banks to calculate the log mel spectrogram. The lower and upper cut-off frequencies of the mel banks are set to 50 Hz and 14 kHz to remove low frequency noise and the aliasing effects. We use the *torchlibrosa*¹, a PyTorch implementation of functions of *librosa* [46] to build log mel spectrogram extraction into PANNs. The log mel spectrogram of a 10-second audio clip has a shape of 1001×64 . The extra one frame is caused by applying the “centre” argument when calculating STFT. A batch size of 32, and an Adam [47] optimizer with a learning rate of 0.001 is used for training. Systems are trained on a single card Tesla-V100-PCIE-32GB. Each system takes around 3 days to train from scratch for 600 k iterations.

B. Evaluation metrics

Mean average precision (mAP), mean area under the curve (mAP) and d-prime are used as official evaluation metrics for AudioSet tagging [20][1]. Average precision (AP) is the

TABLE IV
COMPARISON WITH PREVIOUS METHODS

	mAP	AUC	d-prime
Random guess	0.005	0.500	0.000
Google CNN [1]	0.314	0.959	2.452
Single-level attention [16]	0.337	0.968	2.612
Multi-level attention [17]	0.360	0.970	2.660
Large feature-level attention [20]	0.369	0.969	2.640
TAL Net [19]	0.362	0.965	2.554
DeepRes [48]	0.392	0.971	2.682
Our proposed CNN14	0.431	0.973	2.732

area under the recall and precision curve. AP does not depend on the number of true negatives, because neither precision nor recall depends on the number of true negatives. On the other hand, AUC is the area under the false positive rate and true positive rate (recall) which reflects the influence of the true negatives. The d-prime [1] is also used as an metric and be calculated directly from AUC [1]. All metrics are calculated on individual classes, and then averaged across all classes. Those metrics are also called macro metrics.

C. AudioSet tagging results

1) *Comparison with previous methods*: Table IV shows the comparison of our proposed CNN14 system with previous AudioSet tagging systems. We choose CNN14 as a basic model to investigate a various of hyper-parameter configurations for AudioSet tagging, because CNN14 is a standard CNN that has a simple architecture, and can be compared with previous CNN systems [3][33]. As a baseline, random guess achieves an mAP of 0.005, an AUC of 0.500 and a d-prime of 0.000, respectively. The result released by Google [1] trained with embedding features from [13] achieved an mAP of 0.314 and an AUC of 0.959, respectively. The single-level attention and multi-level attention systems [16], [17] achieved mAPs of 0.337 and 0.360, which were later improved by a feature-level attention neural network that achieved an mAP of 0.369. Wang et al. [19] investigated five different types of attention functions and achieved an mAP of 0.362. All the above systems were built on the embedding features released with AudioSet [1]. The recent DeepRes system [48] was built on waveforms downloaded from YouTube, and achieved an mAP of 0.392. The bottom rows of Table IV shows our proposed

¹<https://github.com/qiuqiangkong/torchlibrosa>

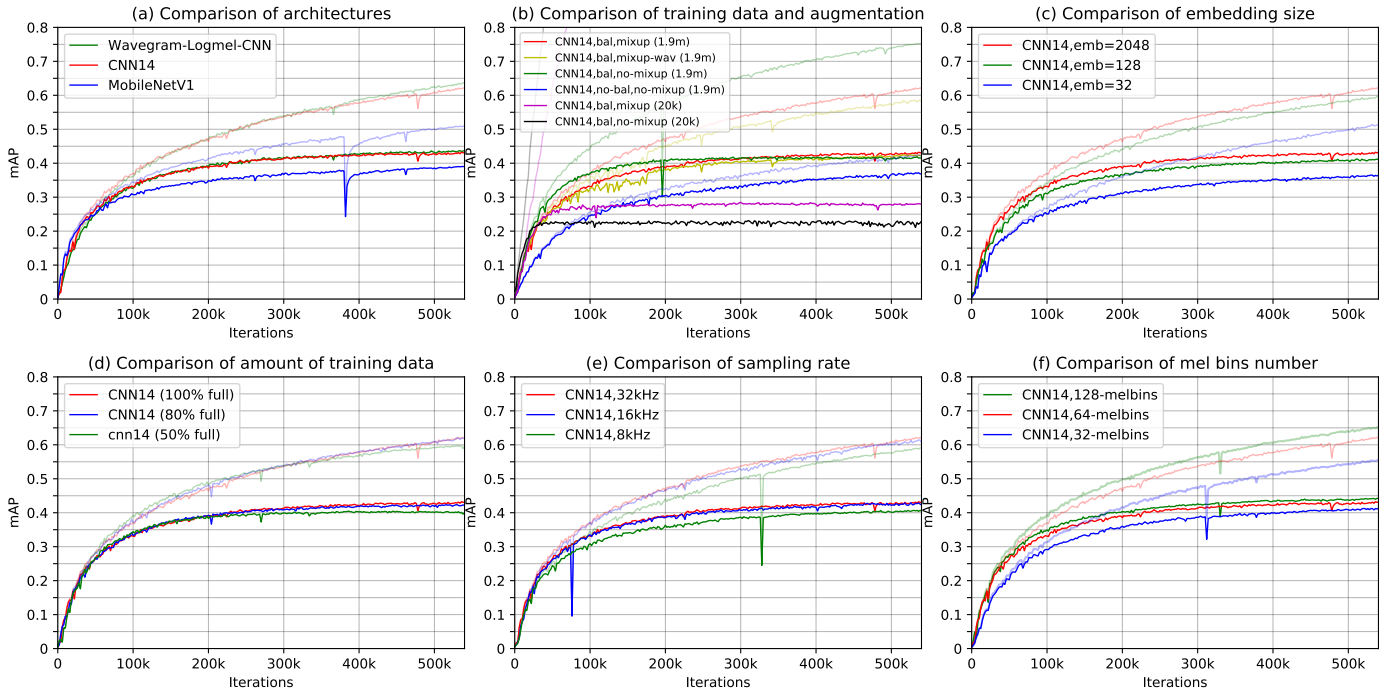


Fig. 4. Results of PANNs on AudioSet tagging. The transparent and solid lines are training mAP and evaluation mAP, respectively. The six plots show the results with different: (a) architectures; (b) data balancing and data augmentation; (c) embedding size; (d) amount of training data; (e) sampling rate; (f) number of mel bins.

CNN14 system achieves an mAP of 0.431, outperforming the best of previous systems. We use CNN14 as a backbone to build Wavegram-Logmel-CNN for fair comparison with the CNN14 system. Fig. 4(a) shows that Wavegram-Logmel-CNN outperforms the CNN14 system, and the MobileNetV1 system. Detailed results are shown in later this section in Table XI.

2) *Class-wise performance*: Fig. 3 shows the class-wise AP of different sound classes with the CNN14 system. The left, middle, right columns show the AP of sound classes with the number of training clips ranked the 1st to 10th, 250th to 260th and 517th to 527th in the training set of AudioSet. The performance of different sound classes can be very different. For example, “Music” and “Speech” achieve APs of over 0.80. On the other hand, some sound classes such as “Inside, small” achieve an AP of only 0.19. Fig. 3 shows that APs are usually not correlated with the number of training clips. For example, the left column shows that “Inside, small” contains 70,159 training clips, while its AP is low. In contrast, the right column shows that “Hoot” only has 106 training clips, but achieves an AP of 0.86, and is larger than many other sound classes with more training clips. In the end of this article, we plot the mAP of all 527 sound classes in Fig. 12, which shows the class-wise comparison of the CNN14, MobileNetV1 and Wavegram-Logmel-CNN systems with previous state-of-the-art audio tagging system [20] built with embedding features released by [1]. The blue bars in Fig. 12 show the number of training clips in logarithmic scale. The “+” symbol indicates label qualities between 0 and 1, which are measured by the percentage of correctly labelled audio clips verified by an expert [1]. The label quality vary from sound class to sound class. The “-” symbol indicates the

TABLE V
RESULTS WITH DATA BALANCING AND AUGMENTATION

Augmentation	mAP	AUC	d-prime
no-bal,no-mixup (20k)	0.224	0.894	1.763
bal,no-mixup (20k)	0.221	0.879	1.652
bal,mixup (20k)	0.278	0.905	1.850
no-bal,no-mixup (1.9m)	0.375	0.971	2.690
bal,no-mixup (1.9m)	0.416	0.968	2.613
bal,mixup (1.9m)	0.431	0.973	2.732
bal,mixup-wav (1.9m)	0.425	0.973	2.720

TABLE VI
RESULTS OF DIFFERENT HOP SIZES

Hop size	Time resolution	mAP	AUC	d-prime
1000	31.25 ms	0.400	0.969	2.645
640	20.00 ms	0.417	0.972	2.711
500	15.63 ms	0.417	0.971	2.682
320	10.00 ms	0.431	0.973	2.732

sound classes whose label quality are not available. Fig. 12 shows that the average precisions of some classes are higher than others. For example, sound classes such as “bagpipes” achieve an average precision of 0.90, while sound classes such as “mouse” achieve an average precision less than 0.2. One explanation is that the audio tagging difficulty is different between sound class to sound class. In addition, audio tagging performance is not always correlated with the number of training clips and label qualities [20]. Fig. 12 shows that our proposed systems outperform previous state-of-the-art systems [16], [17] across a wide range of sound classes.

TABLE VII
RESULTS OF DIFFERENT EMBEDDING DIMENSIONS

Embedding	mAP	AUC	d-prime
32	0.364	0.958	2.437
128	0.412	0.969	2.634
512	0.420	0.971	2.689
2048	0.431	0.973	2.732

TABLE VIII
RESULTS OF PARTIAL TRAINING DATA

Training data	mAP	AUC	d-prime
50% of full	0.406	0.964	2.543
80% of full	0.426	0.971	2.677
100% of full	0.431	0.973	2.732

3) *Data balancing*: Section IV-A introduces the data balancing technique that we use to train AudioSet tagging systems. Fig. 4(b) shows the performance of the CNN14 system with and without data balancing. The blue curve shows that it takes a long time to train PANNs without data balancing. The green curve shows that with data balancing, a system converges faster within limited training iterations. In addition, the systems trained with full 1.9 million training clips perform better than the systems trained with the balanced subset of 20k training clips. Table V shows that the CNN14 system achieves an mAP of 0.416 with data balancing, higher than that without data balancing (0.375).

4) *Data augmentation*: We show that mixup data augmentation plays an important role in training PANNs. By default, we apply mixup on the log mel spectrogram. Fig. 4(b) and Table V shows that the CNN14 system trained with mixup data augmentation achieves an mAP of 0.431, outperforming that trained without mixup data augmentation (0.416). Mixup is especially useful when training with the balanced subset containing only 20k training clips, yielding an mAP of 0.278, compared to training without mixup (0.221). In addition, we show that mixup on the log mel spectrogram achieves an mAP of 0.431, outperforming mixup in the time-domain waveform of 0.425, when training with full training data. This suggests that mixup is more effective when used with the log mel spectrogram than with the time-domain waveform.

5) *Hop sizes*: The hop size is the number of samples between adjacent frames. A small hop size leads to high resolution in the time domain. We investigate the influence of different hop sizes on AudioSet tagging with the CNN14 system. We investigate hop sizes of 1000, 640, 500 and 320: these correspond to time domain resolutions of 31.25 ms, 20.00 ms, 15.63 ms and 10.00 ms between adjacent frames, respectively. Table VI shows that the mAP score increases as hop sizes decrease. With a hop size of 320, the CNN14 system achieves an mAP of 0.431, outperforming the larger hop sizes of 500, 640 and 1000.

6) *Embedding dimensions*: Embedding features are fixed-length vectors that summarize audio clips. By default, the CNN14 has an embedding feature dimension of 2048. We investigate a range of CNN14 systems with embedding dimen-

TABLE IX
RESULTS OF DIFFERENT SAMPLE RATES

Sample rate	mAP	AUC	d-prime
8 kHz	0.406	0.970	2.654
16 kHz	0.427	0.973	2.719
32 kHz	0.431	0.973	2.732

TABLE X
RESULTS OF DIFFERENT MEL BINS

Mel bins	mAP	AUC	d-prime
32 bins	0.413	0.971	2.691
64 bins	0.431	0.973	2.732
128 bins	0.442	0.973	2.735

sions of 32, 128, 512 and 2048. Fig. 4(c) and Table VII show that mAP performance increases as embedding dimension increases.

7) *Training with partial data*: The audio clips of AudioSet are sourced from YouTube. Same audio clips are no longer downloadable, and others may be removed in the future. For better reproducibility of our work in future, we investigate the performance of systems trained with randomly chosen partial data ranging from 50% to 100% of our downloaded data. Fig. 4(d) and Table VIII show that the mAP decreases slightly from 0.431 to 0.426 (a 1.2% drop) when the CNN14 system is trained with 80% of full data, and decreases to 0.406 (a 5.8% drop) when trained with 50% of full data. This result shows that the amount of training data is important for training PANNs.

8) *Sample rate*: Fig. 4(e) and Table IX show the performance of the CNN14 system trained with different sample rate. The CNN14 system trained with 16 kHz audio recordings achieves an mAP of 0.427, similar (within 1.0%) to the CNN14 system trained with a sample rate of 32 kHz. On the other hand, the CNN14 system trained with 8 kHz audio recordings achieves a lower mAP of 0.406 (5.8% lower). This indicates that information in the 4 kHz - 8 kHz range is useful for audio tagging.

9) *Mel bins*: Fig. 4(f) and Table X show the performance of the CNN14 system trained with different number of mel bins. The system achieves an mAP of 0.413 with 32 mel bins, compared to 0.431 with 64 mel bins and 0.442 with 128 mel bins. This result suggests that PANNs achieve better performance with more mel bins, although the computation complexity increases linearly with the number of mel bins. Throughout this paper, we adopt 64 mel bins for extracting the log mel spectrogram, as a trade-off between computational complexity and system performance.

10) *Number of CNN layers*: We investigate the performance of CNN systems with 6, 10 and 14 layers, as described in Section II-A. Table XI shows that the 6-, 10- and 14-layer CNNs achieve mAPs of 0.343, 0.380 and 0.431, respectively. This result suggests that PANNs with deeper CNN architectures achieve better performance than shallower CNN architectures. This result is in contrast to previous audio tagging systems trained on smaller datasets where shallower

TABLE XI
RESULTS OF DIFFERENT SYSTEMS

Architecture	mAP	AUC	d-prime
CNN6	0.343	0.965	2.568
CNN10	0.380	0.971	2.678
CNN14	0.431	0.973	2.732
ResNet22	0.430	0.973	0.270
ResNet38	0.434	0.974	2.737
ResNet54	0.429	0.971	2.675
MobileNetV1	0.389	0.970	2.653
MobileNetV2	0.383	0.968	2.624
DaiNet [31]	0.295	0.958	2.437
LeeNet11 [42]	0.266	0.953	2.371
LeeNet24	0.336	0.963	2.525
Res1dNet31	0.365	0.958	2.444
Res1dNet51	0.355	0.948	2.295
Wavegram-CNN	0.389	0.968	2.612
Wavegram-Logmel-CNN	0.439	0.973	2.720

CNNs such as 9-layer CNN performed better than deeper CNNs [33]. One possible explanation is that smaller datasets may suffer from overfitting, while AudioSet is large enough to train deeper CNNs, at least up to the 14 layers CNNs that we investigate.

11) *ResNets*: We apply ResNets to investigate the performance of deeper PANNs. Table XI shows that the ResNet22 system achieves an mAP of 0.430 similar to the CNN14 system. ResNet38 achieves an mAP of 0.434, slightly outperforming other systems. ResNet54 achieves an mAP of 0.429, which does not further improve the performance.

12) *MobileNets*: The systems mentioned above show that PANNs achieve good performance in AudioSet tagging. However, those systems do not consider computational efficiency when implemented on portable devices. We investigate building PANNs with light weight MobileNets described in Section II-C. Table XI shows that MobileNetV1 achieves an mAP of 0.389, 9.7% lower to the CNN14 system of 0.431. The number of multiplication and addition (multi-adds) and parameters of the MobileNetV1 system are only 8.6% and 5.9% of the CNN14 system, respectively. The MobileNetV2 system achieves an mAP of 0.383, 11.1% lower than CNN14, and is more computationally efficient than MobileNetV1, where the number of multi-adds and parameters are only 6.7% and 5.0% of the CNN14 system.

13) *One-dimensional CNNs*: Table XI shows the performance of one-dimensional CNNs. The DaiNet with 18 layers [31] achieves an mAP of 0.295. The LeeNet11 with 11 layers [42] achieves an mAP of 0.266. Our improved LeeNet with 24 layers improves the mAP of LeeNet11 to 0.336. Our proposed Res1dNet31 and Res1dNet51 described in Section II-D3 achieve mAPs of 0.365 and 0.355 respectively, and achieve state-of-the-art performance among one-dimensional CNN systems.

14) *Wavegram-Logmel-CNN*: The bottom rows of Table XI show the result of our proposed Wavegram-CNN and Wavegram-Logmel-CNN systems. The Wavegram-CNN system achieves an mAP of 0.389, outperforming the best previous one-dimensional CNN system (Res1dNet31). This

TABLE XII
NUMBER OF MULTI-ADDS AND PARAMETERS OF DIFFERENT SYSTEMS

Architecture	Multi-Adds	Parameters
CNN6	21.986×10^9	4,837,455
CNN10	28.166×10^9	5,219,279
CNN14	42.220×10^9	80,753,615
ResNet22	30.081×10^9	63,675,087
ResNet38	48.962×10^9	73,783,247
ResNet54	54.563×10^9	104,318,159
MobileNetV1	3.614×10^9	4,796,303
MobileNetV2	2.810×10^9	4,075,343
DaiNet	30.395×10^9	4,385,807
LeeNet11	4.741×10^9	748,367
LeeNet24	26.369×10^9	10,003,791
Res1dNet31	32.688×10^9	80,464,463
Res1dNet51	61.833×10^9	106,538,063
Wavegram-CNN	44.234×10^9	80,991,759
Wavegram-Logmel-CNN	53.510×10^9	81,065,487

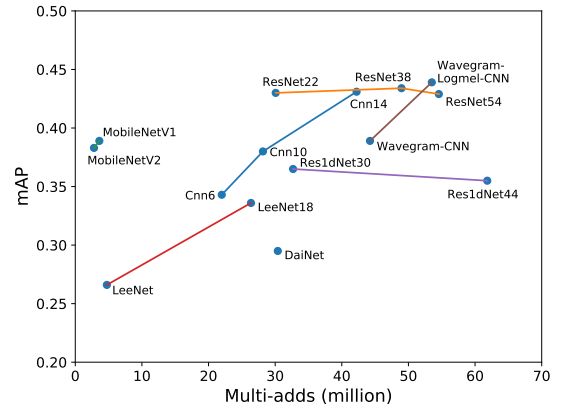


Fig. 5. Multi-adds versus mAP of AudioSet tagging systems. The same types of architectures are grouped in the same color.

indicates that the Wavegram is an effective learnt feature. Furthermore, our proposed Wavegram-Logmel-CNN system achieves a state-of-the-art mAP of 0.439 among all PANNs.

15) *Complexity analysis*: We analyze the computational complexity of PANNs for inference. The number of multi-adds and parameters are two important factors for complexity analysis. The middle column of Table XII shows the number of multi-adds to infer a 10-second audio clip. The right column of Table XII shows the number of parameters of different systems. The number of multi-adds and parameters of the CNN14 system are 42.2×10^9 and 80.8 million, respectively, which are larger than the CNN6 and CNN10 systems. The number of multi-adds for the ResNets22 and ResNet38 systems are slightly less than for the CNN14 system. The ResNet54 system contains the most multi-adds at 54.6×10^9 . One-dimensional CNNs have a similar computational cost to the two-dimensional CNNs. The best performing one-dimensional system, Res1dNet31, contains 32.7×10^9 multi-adds and 80.5 million parameters. Our proposed Wavegram-CNN system contains 44.2×10^9 multi-adds and 81.0 million parameters, which is similar to CNN14. The Wavegram-Logmel-CNN system slightly increases the multi-adds to 53.5×10^9 , and the number of parameters is to 81.1 million, which is similar to CNN14. To reduce the number of multi-adds and parameters,

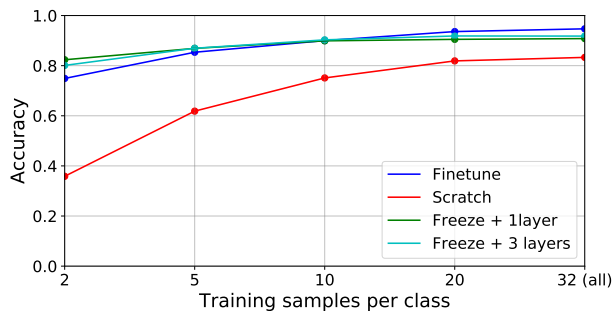


Fig. 6. Accuracy of ESC-50 with various number of training clips per class.

TABLE XIII
ACCURACY OF ESC-50

	STOA [49]	Scratch	fine-tune	Freeze_L1	Freeze_L3
Acc.	0.865	0.833	0.947	0.908	0.918

MobileNets are applied. The MobileNetV1 and MobileNetV2 systems are light weight CNNs, with only 3.6×10^9 and 2.8×10^9 multi-adds and around 4.8 million and 4.1 million parameters, respectively. MobileNets reduce both the computational cost and system size. Figure 5 summarizes the mAP versus multi-adds of different PANNs. The same type of systems are linked by lines of the same color. The mAP increases from bottom to top. On the top-right is our proposed Wavegram-Logmel-CNN system that achieves the best mAP. On the top-left are MobileNetV1 and MobileNetV2 that are the most computationally efficient systems.

D. Transfer to other tasks

In this section, we investigate the application of PANNs to a range of other pattern recognition tasks. PANNs can be useful for few-shot learning, for the tasks where only a limited number of training clips are provided. Few-shot learning is an important research topic in audio pattern recognition, as collecting labelled data can be time consuming. We transfer PANNs to other audio pattern recognition tasks using the methods described in Section V. To begin with, we resample all audio recordings to 32 kHz, and convert them to monophonic to be consistent with the PANNs trained on AudioSet. We perform the following strategies described in Section V for each task: 1) Train a system from scratch; 2) Use a PANN as a feature extractor; 3) Fine-tune a PANN. When using a PANN as the feature extractor, we build classifiers on the embedding features with one and three fully-connected layers, and call them “Freeze + 1 layers” (Freeze_L1) and “Freeze + 3 layers” (Freeze_L3), respectively. We adopt the CNN14 system for transfer learning to provide a fair comparison with other CNN based systems for audio pattern recognition. We also investigate the performance of PANNs trained with different number of shots when training other audio pattern recognition tasks.

1) *ESC-50*: ESC-50 is an environmental sound dataset [50] consisting of 50 sound events, such as “Dog” and “Rain”. There are 2,000 5-second audio clips in the dataset, with

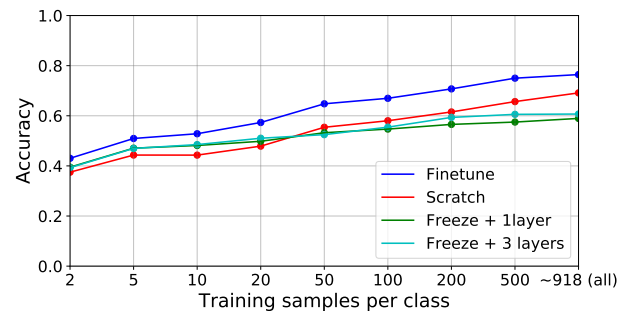


Fig. 7. Accuracy of DCASE 2019 Task 1 with various number of training clips per class.

TABLE XIV
ACCURACY OF DCASE 2019 TASK 1

	STOA [51]	Scratch	Fine-tune	Freeze_L1	Freeze_L3
Acc.	0.851	0.691	0.764	0.589	0.607

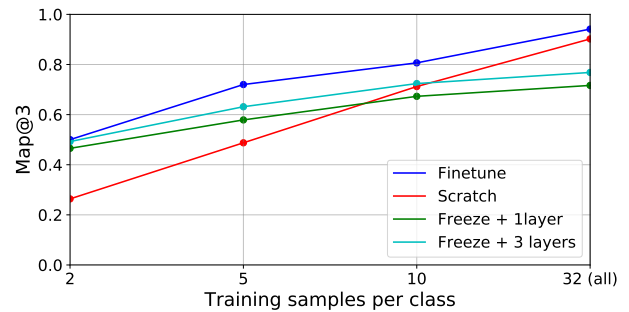


Fig. 8. Accuracy of DCASE 2018 Task 2 with various number of training clips per class.

TABLE XV
ACCURACY OF DCASE 2018 TASK 2

	STOA [52]	Scratch	Fine-tune	Freeze_L1	Freeze_L3
mAP@3	0.954	0.902	0.941	0.717	0.768

40 clips per class. Table XIII shows the 5-fold cross validation [50] accuracy of the CNN14 system. Sailor et al. [49] proposed a state-of-the-art system for ESC-50, achieved an accuracy of 0.865 using unsupervised filterbank learning with a convolutional restricted Boltzmann machine. Our fine-tuned system achieves an accuracy of 0.947, outperforming previous state-of-the-art system by a large margin. The Freeze_L1 and Freeze_L3 systems achieve accuracies of 0.918 and 0.908, respectively. Training the CNN14 system from scratch achieves an accuracy of 0.833. Fig. 6 shows the accuracy of ESC-50 with different numbers of training clips of each sound class. Using a PANN as a feature extractor achieves the best performance when fewer than 10 clips per sound class are available for training. With more training clips, the fine-tuned systems achieve better performance. Both the fine-tuned system and the system using the PANN as a feature extractor outperform the systems trained from scratch.

2) *DCASE 2019 Task 1*: DCASE 2019 Task 1 is an acoustic scene classification task [2], with a dataset consisting of over

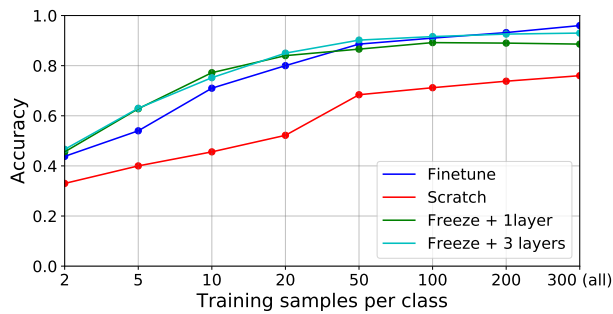


Fig. 9. Accuracy of MSoS with various number of training clips per class.

TABLE XVI
ACCURACY OF MSoS

	STOA [53]	Scratch	Fine-tune	Freeze_L1	Freeze_L3
Acc.	0.930	0.760	0.960	0.886	0.930

40 hours of stereo recordings collected from various acoustic scenes in 12 European cities. We focus on Subtask A, where each audio recording has two channels with a sampling rate of 48 kHz. In the development set, there are 9185 and 4185 audio clips for training and validation respectively. We convert the stereo recordings to monophonic by averaging the stereo channels. CNN14 trained from scratch achieves an accuracy of 0.691, while the fine-tuned system achieves an accuracy of 0.764. Freeze_L1 and Freeze_L3 achieve accuracies of 0.689 and 0.607 respectively, and do not outperform the CNN14 trained from scratch. One possible explanation for this underperformance is that the audio recordings of acoustic scene classification have different distribution of AudioSet. So using PANN as a feature extractor does not outperform fine-tune or train a system from scratch. The fine-tuned system achieves better performance than the system trained from scratch. Fig. 7 shows the classification accuracy of systems with various numbers of training clips per class. Table XIV shows the overall performance. The state-of-the-art system of Chen et al [51]. achieves an accuracy of 0.851 using the combination of various classifiers and stereo recordings as input, while we do not use any ensemble methods and stereo recordings.

3) *DCASE 2018 Task 2*: DCASE 2018 Task 2 is a general-purpose automatic audio tagging task [54] using a dataset of audio recordings from Freesound annotated with a vocabulary of 41 labels from the AudioSet ontology. The development set consists of 9,473 audio recordings with durations from 300 ms to 30 s. The mAP@3 is used for evaluating system performance [54]. In training, we break or pad audio recordings into 4-second audio segments. In inference, we average the predictions of those segments to obtain the prediction of an audio recording. Table XV shows that the best previous method proposed by Jeong and Lim [52] achieves an mAP@3 of 0.954 using an ensemble of several systems. In comparison, our CNN14 system trained from scratch achieves an accuracy of 0.902. The fine-tuned CNN14 system achieves an mAP@3 of 0.941. The Freeze_L1 and Freeze_L3 systems achieve accuracies of 0.717 and 0.768 respectively. Fig. 8 shows the

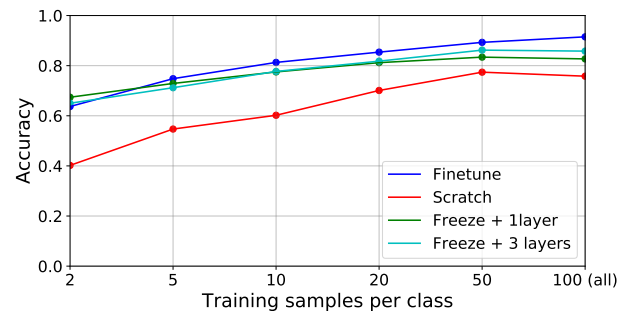


Fig. 10. Accuracy of GTZAN with various number of training clips per class.

TABLE XVII
ACCURACY OF GTZAN

	STOA [56]	Scratch	Fine-tune	Freeze_L1	Freeze_L3
Acc.	0.939	0.758	0.915	0.827	0.858

mAP@3 with different numbers of training clips. The fine-tuned CNN14 system outperforms the systems trained from scratch and the systems using PANN as a feature extractor. The fine-tuned CNN14 system achieves comparable results to the state-of-the-art system.

4) *MSoS*: The Making Sense of Sounds (MSoS) data challenge [55] is a task to predict an audio recording to one of five categories: “Nature”, “Music”, “Human”, “Effects” and “Urban”. The dataset consists of a development set of 1,500 audio clips and an evaluation set of 500 audio clips. All audio clips have a duration of 4 seconds. The state-of-the-art system proposed by Chen and Gupta [53] achieves an accuracy of 0.930. Our fine-tuned CNN14 achieves an accuracy of 0.960, outperforming previous state-of-the-art system. CNN14 trained from scratch achieves an accuracy of 0.760. Fig. 9 shows the accuracy of the systems with different number of training clips. The fine-tuned CNN14 system and the system using CNN14 as a feature extractor outperforms CNN14 trained from scratch.

5) *GTZAN*: The GTZAN dataset [57] is a music genre classification dataset containing 1,000 30-second music clips of 10 genres of music, such as “Classical” and “Country”. All music clips have a duration of 30 seconds and a sampling rate of 22,050 Hz. In development, 10-fold cross validation is used to evaluate the performance of systems. Table XVII shows that previous state-of-the-art system proposed by Liu et al. [56] achieves an accuracy of 0.939 using a bottom-up broadcast neural network. The fine-tuned CNN14 system achieves an accuracy of 0.915, outperforming CNN14 trained from scratch with an accuracy of 0.758 and the Freeze_L1 and Freeze_L3 systems with accuracies of 0.827 and 0.858 respectively. Fig. 10 shows the accuracy of systems with different numbers of training clips. The Freeze_L1 and Freeze_L3 systems achieve better performance than other systems trained with less than 10 clips per class. With more training clips, the fine-tuned CNN14 system performs better than other systems.

6) *RAVDESS*: The Ryerson Audio-Visual Database of Emotional Speech and Song (RAVDESS) is a human speech emotion dataset [59]. The database consists of sounds from

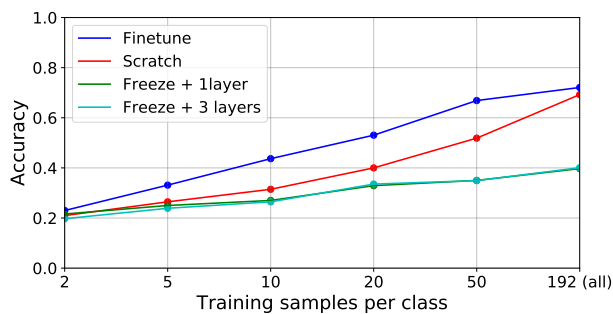


Fig. 11. Accuracy of RAVDESS with various number of training clips per class.

TABLE XVIII
ACCURACY OF RAVDESS

	STOA [58]	Scratch	Fine-tune	Freeze_L1	Freeze_L3
Acc.	0.645	0.692	0.721	0.397	0.401

24 professional actors including 12 female and 12 male simulating 8 emotions, such as “Happy” and “Sad”. The task is to classify each sound clip into an emotion. There are 1,440 audio clips in the development set. We evaluate our systems with 4-fold cross validation. Table XVIII shows that previous state-of-the-art system proposed by Zeng et al. [58] achieves an accuracy of 0.645. Our CNN14 system trained from scratch achieves an accuracy of 0.692. The fine-tuned CNN14 system achieves a state-of-the-art accuracy of 0.721. The Freeze_L1 and Freeze_L3 systems achieve much lower accuracies of 0.397 and 0.401 respectively. Fig. 11 shows the accuracy of systems with respect to a range of training clips. The fine-tuned systems and the system trained from scratch outperform the system using PANN as a feature extractor. This suggests that audio recordings of the RAVDESS dataset may have different distributions of the AudioSet dataset. Therefore, the parameters of a PANN need be fine-tuned to achieve good performance on the RAVDESS classification task.

E. Discussion

In this article, we have investigated a wide range of PANNs for AudioSet tagging. Several of our proposed PANNs have outperformed previous state-of-the-art AudioSet tagging systems, including CNN14 achieves an mAP of 0.431, and ResNet38 achieves an mAP of 0.434, outperforming Google’s baseline of 0.314. MobileNets are light-weight systems that have fewer multi-adds and numbers of parameters. MobileNetV1 achieves an mAP of 0.389. Our adapted one-dimensional system Res1dNet31 achieves an mAP of 0.365, outperforming previous one-dimensional CNNs including DaiNet [31] of 0.295 and LeeNet11 [42] of 0.266. Our proposed Wavegram-Logmel-CNN system achieves the highest mAP of 0.439 among all PANNs. PANNs can be used as a pretrained model for new audio pattern recognition tasks.

PANNs trained on the AudioSet dataset were transferred to six audio pattern recognition tasks. We show that fine-tuned PANNs achieve state-of-the-art performance in the ESC-50, MSOS and RAVDESS classification tasks, and approach the

state-of-the-art performance in the DCASE 2018 Task 2 and the GTZAN classification task. Of the PANN systems, the fine-tuned PANNs always outperform PANNs trained from scratch on new tasks. The experiments show that PANNs have been successful in generalizing to other audio pattern recognition tasks with limited number of training data.

VII. CONCLUSION

We have presented pretrained audio neural networks (PANNs) trained on the AudioSet for audio pattern recognition. A wide range of neural networks are investigated to build PANNs. We propose a Wavegram feature learnt from waveform, and a Wavegram-Logmel-CNN that achieves state-of-the-art performance in AudioSet tagging, achieving an mAP of 0.439. We also investigate the computational complexity of PANNs. We show that PANNs can be transferred to a wide range of audio pattern recognition tasks and outperform several previous state-of-the-art systems. PANNs can be useful when fine-tuned on a small amount of data on new tasks. In the future, we will extend PANNs to more audio pattern recognition tasks.

REFERENCES

- [1] J. F. Gemmeke, D. P. Ellis, D. Freedman, A. Jansen, W. Lawrence, R. C. Moore, M. Plakal, and M. Ritter, “Audio Set: An ontology and human-labeled dataset for audio events,” in *IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, 2017, pp. 776–780.
- [2] A. Mesaros, T. Heittola, and T. Virtanen, “A multi-device dataset for urban acoustic scene classification,” in *Workshop on Detection and Classification of Acoustic Scenes and Events (DCASE)*, 2018, pp. 9–13.
- [3] K. Choi, G. Fazekas, and M. Sandler, “Automatic tagging using deep convolutional neural networks,” in *Conference of the International Society for Music Information Retrieval (ISMIR)*, 2016, pp. 805–811.
- [4] E. Cakir, T. Heittola, H. Huttunen, and T. Virtanen, “Polyphonic sound event detection using multi label deep neural networks,” in *International Joint Conference on Neural Networks (IJCNN)*, 2015.
- [5] J. P. Woodard, “Modeling and classification of natural sounds by product code hidden Markov models,” *IEEE Transactions on Signal Processing*, vol. 40, pp. 1833–1835, 1992.
- [6] D. P. W. Ellis, “Detecting alarm sounds,” <https://academiccommons.columbia.edu/doi/10.7916/D8F19821/download>, 2001.
- [7] D. Stowell, D. Giannoulis, E. Benetos, M. Lagrange, and M. D. Plumbley, “Detection and classification of acoustic scenes and events,” *IEEE Transactions on Multimedia*, vol. 17, pp. 1733–1746, 2015.
- [8] A. Mesaros, T. Heittola, E. Benetos, P. Foster, M. Lagrange, T. Virtanen, and M. D. Plumbley, “Detection and classification of acoustic scenes and events: Outcome of the DCASE 2016 challenge,” *IEEE/ACM Transactions on Audio, Speech and Language Processing (TASLP)*, vol. 26, pp. 379–393, 2018.
- [9] A. Mesaros, T. Heittola, A. Diment, B. Elizalde, A. Shah, E. Vincent, B. Raj, and T. Virtanen, “DCASE 2017 challenge setup: Tasks, datasets and baseline system,” in *Workshop on Detection and Classification of Acoustic Scenes and Events (DCASE)*, 2017, pp. 85–92.
- [10] “DCASE Challenge 2019,” <http://dcase.community/challenge2019>, 2019.
- [11] J. Deng, W. Dong, R. Socher, L.-J. Li, K. Li, and L. Fei-Fei, “ImageNet: A large-scale hierarchical image database,” in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2009, pp. 248–255.
- [12] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, “BERT: Pre-training of deep bidirectional transformers for language understanding,” in *Annual Conference of the North American Chapter of the Association for Computational Linguistics (NAACL)*, 2018, pp. 4171–4186.
- [13] S. Hershey, S. Chaudhuri, D. P. Ellis, J. F. Gemmeke, A. Jansen, R. C. Moore, M. Plakal, D. Platt, R. A. Saurous, B. Seybold et al., “CNN architectures for large-scale audio classification,” in *IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, 2017, pp. 131–135.

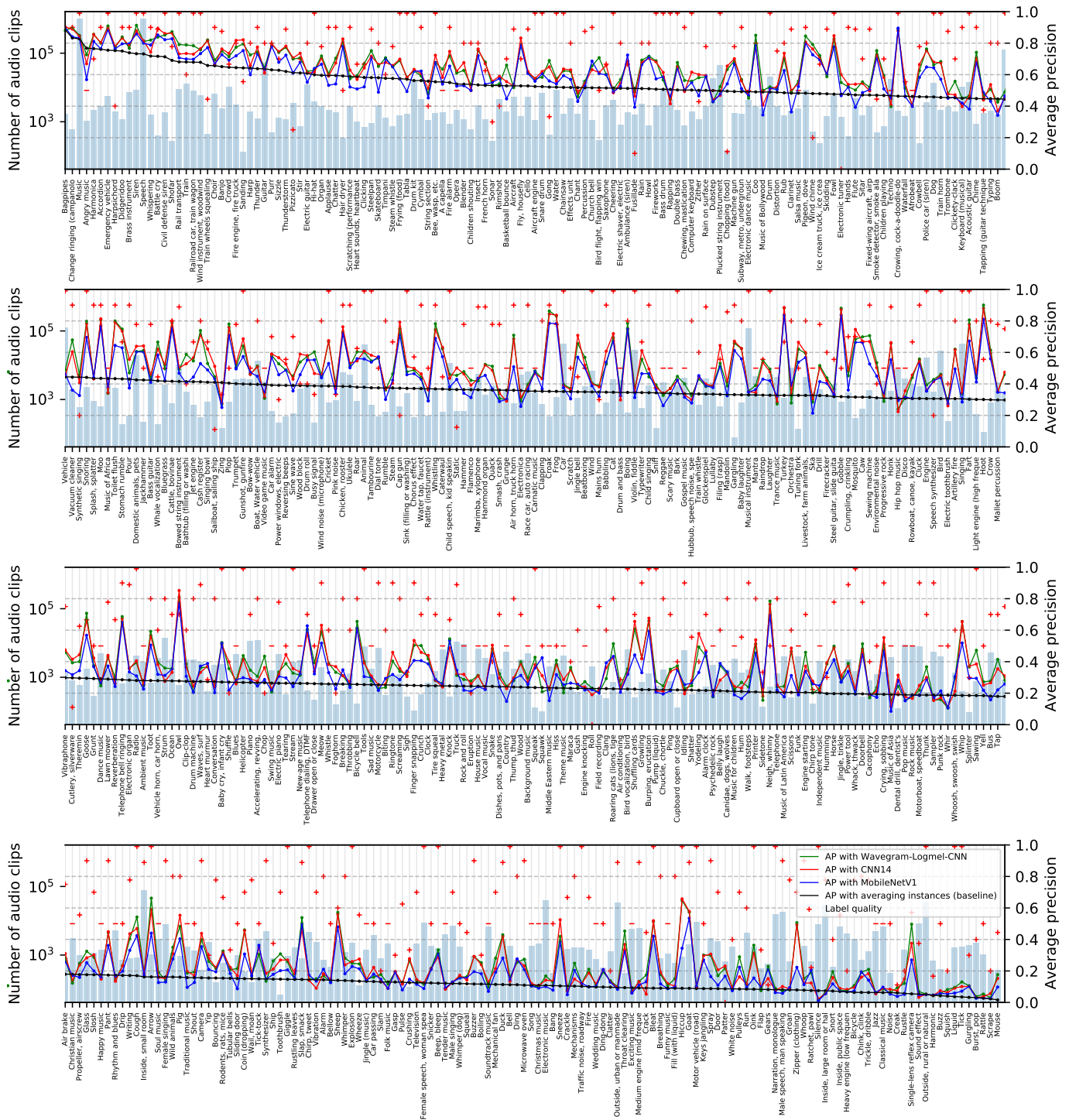
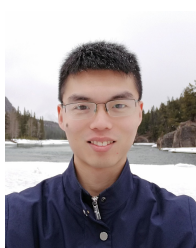


Fig. 12. Class-wise performance of AudioSet tagging systems. Red, blue and black curves are APs of CNN14, MobileNetV1 and the audio tagging system [20]. The blue bars show the number of training clips in logarithmic scale.

- [14] K. Choi, G. Fazekas, M. Sandler, and K. Cho, "Transfer learning for music classification and regression tasks," in *Conference of the International Society of Music Information Retrieval (ISMIR)*, 2017, pp. 141–149.
- [15] J. Pons, O. Nieto, M. Prockup, E. Schmidt, A. Ehmann, and X. Serra, "End-to-end learning for music audio tagging at scale," in *Conference of the International Society for Music Information Retrieval (ISMIR)*, 2017, pp. 637–644.
- [16] Q. Kong, Y. Xu, W. Wang, and M. D. Plumbley, "Audio Set classification with attention model: A probabilistic perspective," in *IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, 2018, pp. 316–320.
- [17] C. Yu, K. S. Barsim, Q. Kong, and B. Yang, "Multi-level attention model for weakly supervised audio classification," in *Detection and Classification of Acoustic Scenes and Events (DCASE)*, 2018, pp. 188–192.
- [18] S.-Y. Chou, J.-S. R. Jang, and Y.-H. Yang, "Learning to recognize transient sound events using attentional supervision," in *International Joint Conferences on Artificial Intelligence (IJCAI)*, 2018, pp. 3336–3342.
- [19] Y. Wang, J. Li, and F. Metzger, "A comparison of five multiple instance learning pooling functions for sound event detection with weak labeling,"

- in *IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, 2019, pp. 31–35.
- [20] Q. Kong, C. Yu, Y. Xu, T. Iqbal, W. Wang, and M. D. Plumbley, “Weakly labelled audioset tagging with attention neural networks,” *IEEE/ACM Transactions on Audio, Speech, and Language Processing*, vol. 27, pp. 1791–1802, 2019.
- [21] A. Van Den Oord, S. Dieleman, and B. Schrauwen, “Transfer learning by supervised pre-training for audio-based music classification,” in *Conference of the International Society for Music Information Retrieval (ISMIR)*, 2014, pp. 29–34.
- [22] Y. Wang, “Polyphonic sound event detection with weak labeling,” *PhD thesis, Carnegie Mellon University*, 2018.
- [23] E. Law and L. Von Ahn, “Input-agreement: a new mechanism for collecting data using human computation games,” in *Proceedings of the SIGCHI Conference on Human Factors in Computing Systems*, 2009, pp. 1197–1206.
- [24] A. Mesaros, T. Heittola, and T. Virtanen, “TUT database for acoustic scene classification and sound event detection,” in *Conference on European Signal Processing Conference (EUSIPCO)*, 2016, pp. 1128–1132.
- [25] J. Pons and X. Serra, “MUSICNN: Pre-trained convolutional neural networks for music audio tagging,” *arXiv preprint arXiv:1909.06654*, 2019.
- [26] A. Diment and T. Virtanen, “Transfer learning of weakly labelled audio,” in *IEEE Workshop on Applications of Signal Processing to Audio and Acoustics (WASPAA)*, 2017, pp. 6–10.
- [27] D. Li, I. K. Sethi, N. Dimitrova, and T. McGee, “Classification of general audio data for content-based retrieval,” *Pattern Recognition Letters*, vol. 22, pp. 533–544, 2001.
- [28] L. Vuegen, B. Broeck, P. Karsmakers, J. F. Gemmeke, B. Vanrumste, and H. Hamme, “An MFCC-GMM approach for event detection and classification,” in *IEEE Workshop on Applications of Signal Processing to Audio and Acoustics (WASPAA)*, 2013.
- [29] A. Mesaros, T. Heittola, A. Eronen, and T. Virtanen, “Acoustic event detection in real life recordings,” in *European Signal Processing Conference (EUSIPCO)*, 2010, pp. 1267–1271.
- [30] B. Uzkent, B. D. Barkana, and H. Cevikalp, “Non-speech environmental sound classification using SVMs with a new set of features,” *International Journal of Innovative Computing, Information and Control*, vol. 8, pp. 3511–3524, 2012.
- [31] W. Dai, C. Dai, S. Qu, J. Li, and S. Das, “Very deep convolutional neural networks for raw waveforms,” in *IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, 2017, pp. 421–425.
- [32] K. He, X. Zhang, S. Ren, and J. Sun, “Deep residual learning for image recognition,” in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2016, pp. 770–778.
- [33] Q. Kong, Y. Cao, T. Iqbal, Y. Xu, W. Wang, and M. D. Plumbley, “Cross-task learning for audio tagging, sound event detection and spatial localization: DCASE 2019 baseline systems,” *arXiv preprint arXiv:1904.03476*, 2019.
- [34] A. Krizhevsky, I. Sutskever, and G. E. Hinton, “ImageNet classification with deep convolutional neural networks,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2012, pp. 1097–1105.
- [35] K. Simonyan and A. Zisserman, “Very deep convolutional networks for large-scale image recognition,” *International Conference on Learning Representations (ICLR)*, 2015.
- [36] S. Ioffe and C. Szegedy, “Batch normalization: Accelerating deep network training by reducing internal covariate shift,” in *International Conference on Machine Learning (ICML)*, 2015, pp. 448–456.
- [37] V. Nair and G. E. Hinton, “Rectified linear units improve restricted boltzmann machines,” in *International Conference on Machine Learning (ICML)*, 2010, pp. 807–814.
- [38] N. Srivastava, G. Hinton, A. Krizhevsky, I. Sutskever, and R. Salakhutdinov, “Dropout: A simple way to prevent neural networks from overfitting,” *The Journal of Machine Learning Research*, vol. 15, pp. 1929–1958, 2014.
- [39] M. Lin, Q. Chen, and S. Yan, “Network in network,” in *International Conference on Learning Representations (ICLR)*, 2014.
- [40] A. G. Howard, M. Zhu, B. Chen, D. Kalenichenko, W. Wang, T. Weyand, M. Andreetto, and H. Adam, “Mobilenets: Efficient convolutional neural networks for mobile vision applications,” *arXiv preprint arXiv:1704.04861*, 2017.
- [41] M. Sandler, A. Howard, M. Zhu, A. Zhmoginov, and L.-C. Chen, “MobileNetV2: Inverted residuals and linear bottlenecks,” in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2018, pp. 4510–4520.
- [42] J. Lee, J. Park, K. L. Kim, and J. Nam, “Sample-level deep convolutional neural networks for music auto-tagging using raw waveforms,” in *Sound and Music Computing Conference*, 2017, pp. 220–226.
- [43] J. Salamon, C. Jacoby, and J. P. Bello, “A dataset and taxonomy for urban sound research,” in *Proceedings of the ACM International Conference on Multimedia*, 2014, pp. 1041–1044.
- [44] H. Zhang, M. Cisse, Y. N. Dauphin, and D. Lopez-Paz, “mixup: Beyond empirical risk minimization,” in *International Conference on Learning Representations (ICLR)*, 2018.
- [45] D. S. Park, W. Chan, Y. Zhang, C.-C. Chiu, B. Zoph, E. D. Cubuk, and Q. V. Le, “SpecAugment: A simple data augmentation method for automatic speech recognition,” in *INTERSPEECH*, 2019.
- [46] B. McFee, C. Raffel, D. Liang, D. P. Ellis, M. McVicar, E. Battenberg, and O. Nieto, “librosa: Audio and music signal analysis in python,” in *Proceedings of the Python in Science Conference*, vol. 8, 2015, pp. 18–25.
- [47] D. P. Kingma and J. Ba, “Adam: A method for stochastic optimization,” in *International Conference on Learning Representations (ICLR)*, 2015.
- [48] L. Ford, H. Tang, F. Grondin, and J. Glass, “A deep residual network for large-scale acoustic scene analysis,” *INTERSPEECH*, pp. 2568–2572, 2019.
- [49] H. B. Sailor, D. M. Agrawal, and H. A. Patil, “Unsupervised filterbank learning using convolutional restricted Boltzmann machine for environmental sound classification,” in *INTERSPEECH*, 2017, pp. 3107–3111.
- [50] K. J. Piczak, “ESC: Dataset for environmental sound classification,” in *ACM International Conference on Multimedia*, 2015, pp. 1015–1018.
- [51] H. Chen, Z. Liu, Z. Liu, P. Zhang, and Y. Yan, “Integrating the data augmentation scheme with various classifiers for acoustic scene modeling,” *DCASE2019 Challenge, Tech. Rep.*, 2019.
- [52] I.-Y. Jeong and H. Lim, “Audio tagging system for DCASE 2018: focusing on label noise data augmentation and its efficient learning,” *DCASE Challenge Tech. Rep.*, 2018.
- [53] T. Chen and U. Gupta, “Attention-based convolutional neural network for audio event classification with feature transfer learning,” https://cvssp.org/projects/making_sense_of_sounds/site/assets/challenge_abstracts_and_figures/Tianxiang_Chen.pdf, 2018.
- [54] E. Fonseca, M. Plakal, F. Font, D. P. W. Ellis, X. Favory, J. Pons, and X. Serra, “General-purpose tagging of freesound audio with audioset labels: Task description, dataset, and baseline,” in *Workshop on Detection and Classification of Acoustic Scenes and Events (DCASE)*, November 2018, pp. 69–73.
- [55] C. Kroos, O. Bones, Y. Cao, L. Harris, P. J. Jackson, W. J. Davies, W. Wang, T. J. Cox, and M. D. Plumbley, “Generalisation in environmental sound classification: The ‘Making Sense of Sounds’ data set and challenge,” in *IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, 2019, pp. 8082–8086.
- [56] C. Liu, L. Feng, G. Liu, H. Wang, and S. Liu, “Bottom-up broadcast neural network for music genre classification,” *arXiv preprint arXiv:1901.08928*, 2019.
- [57] G. Tzanetakis and P. Cook, “Musical genre classification of audio signals,” *IEEE Transactions on Speech and Audio Processing*, vol. 10, pp. 293–302, 2002.
- [58] Y. Zeng, H. Mao, D. Peng, and Z. Yi, “Spectrogram based multi-task audio classification,” *Multimedia Tools and Applications*, vol. 78, pp. 3705–3722, 2019.
- [59] S. R. Livingstone, K. Peck, and F. A. Russo, “RAVDESS: The Ryerson audio-visual database of emotional speech and song,” in *Annual Meeting of the Canadian Society for Brain, Behaviour and Cognitive Science (CSBCCS)*, 2012, pp. 1459–1462.



Qiugiang Kong (S'17) received the B.Sc. degree in 2012, and the M.E. degree in 2015 from South China University of Technology, Guangzhou, China. He received the Ph.D. degree from University of Surrey, Guildford, UK in 2020. Following his PhD, he joined ByteDance AI Lab as a research scientist. His research topic includes the classification, detection and separation of general sounds and music. He is known for developing attention neural networks for audio tagging, and winning the audio tagging task in the detection and classification of acoustic

scenes and events (DCASE) challenge in 2017. He was nominated as the postgraduate research student of the year in University of Surrey, 2019. He is a frequent reviewer for journals and conferences in the area including IEEE/ACM Transactions on Audio, Speech, and Language Processing.



Yin Cao (M'18) received the B.Sc. degree in Electronic Science and Engineering from Nanjing University, China in 2008, and Ph.D. degree from Institute of Acoustics, Chinese Academy of Sciences, China in 2013. He then worked in Acoustics group at Brigham Young University, US, and at Institute of Acoustics, Chinese Academy of Sciences, China. In 2018, he joined Centre for Vision, Speech and Signal Processing at the University of Surrey. His research topic includes active noise control, air acoustics and signal processing, detection, classification and

separation of audio. He is known for his work on decentralized active noise control, weighted spatial gradients control metric, and polyphonic sound event detection and localization. He was the winner of urban sound tagging in detection and classification of acoustic scenes and events (DCASE) 2020 challenge and achieved second-best of sound event detection and localization tasks in DCASE 2019 challenge. He has served as an Associate Editor for Noise Control Engineering Journal since 2020. He is also a frequent reviewer for IEEE/ACM Transactions on Audio, Speech, and Language Processing.



Turab Iqbal received the B.Eng. degree in Electronic Engineering from the University of Surrey, U.K., in 2017. Currently, he is working towards a Ph.D. degree from the Centre for Vision, Speech and Signal Processing (CVSSP) in the University of Surrey. During his time as a Ph.D. student, he has worked on a number of projects in the area of audio classification and localization using machine learning methods. His research is mainly focused on learning with weakly-labeled or noisy training data.



Wenwu Wang (M'02-SM'11) was born in Anhui, China. He received the B.Sc. degree in 1997, the M.E. degree in 2000, and the Ph.D. degree in 2002, all from Harbin Engineering University, China. He then worked in King's College London, Cardiff University, Tao Group Ltd. (now Antix Labs Ltd.), and Creative Labs, before joining University of Surrey, UK, in May 2007, where he is currently a professor in signal processing and machine learning, and a Co-Director of the Machine Audition Lab within the Centre for Vision Speech and Signal Processing.

He has been a Guest Professor at Qingdao University of Science and Technology, China, since 2018. His current research interests include blind signal processing, sparse signal processing, audio-visual signal processing, machine learning and perception, machine audition (listening), and statistical anomaly detection. He has (co)-authored over 200 publications in these areas. He served as an Associate Editor for IEEE Transactions on Signal Processing from 2014 to 2018. He is also Publication Co-Chair for ICASSP 2019, Brighton, UK. He currently serves as Senior Area Editor for IEEE Transactions on Signal Processing and an Associate Editor for IEEE/ACM Transactions on Audio Speech and Language Processing.



Mark D. Plumbley (S'88-M'90-SM'12-F'15) received the B.A.(Hons.) degree in electrical sciences and the Ph.D. degree in neural networks from University of Cambridge, Cambridge, U.K., in 1984 and 1991, respectively. Following his PhD, he became a Lecturer at King's College London, before moving to Queen Mary University of London in 2002. He subsequently became Professor and Director of the Centre for Digital Music, before joining the University of Surrey in 2015 as Professor of Signal Processing. He is known for his work on analysis

and processing of audio and music, using a wide range of signal processing techniques, including matrix factorization, sparse representations, and deep learning. He is a co-editor of the recent book on Computational Analysis of Sound Scenes and Events, and Co-Chair of the recent DCASE 2018 Workshop on Detection and Classifications of Acoustic Scenes and Events. He is a Member of the IEEE Signal Processing Society Technical Committee on Signal Processing Theory and Methods, and a Fellow of the IET and IEEE.